

Dossier de spécification PFR 2025 – 2026

**Université Paul Sabatier – Faculté des sciences de
l'ingénieur**

Groupe 5 : Delhem Sebastien, Gerain Antonin, Haegel
Lucas, Nosel Adrien, Raharitsifa Ny Aina.

Client : Ferrané Isabelle



Sommaire

Introduction.....	3
I. Présentation du projet.....	4
I.1 – Objectifs du document.....	4
I.2 – Objectifs du projet.....	4
II. Fonctionnalités.....	7
II.1 – Conversions de requêtes (opérateur → système).....	7
II.2 – Simulation des déplacements.....	10
II.3 – Interface utilisateur.....	12
II.4 – Traitement d'images.....	15
III. Organisation.....	18
III.1 – Outils utilisés.....	18
III.2 – Planning prévisionnel.....	19
III.3 – Répartition des rôles et des responsabilités..	20

Introduction

Dans le cadre de la première année de formation en Systèmes Robotiques et Industriels (SRI) à l'UPSSITECH de Toulouse, nous participons au Projet Fil Rouge (PFR). Nos professeur(e)s référent(e)s : Isabelle Ferrané, Julien Piquier et Julien Vanderstraeten sont les clients qui nous suivent tout au long de l'année. Le PFR permet de mobiliser et de combiner les compétences techniques et organisationnelles de chaque membre de l'équipe pour mener ce projet à la réussite.

Le projet nous offre aussi une opportunité de développer nos aptitudes en gestion de projet, en répondant aux attentes et exigences d'un client réel. Notre équipe est composée de cinq membres : Delhem Sébastien, Gerain Antonin, Haegel Lucas, Nosel Adrien et Raharitsifa Ny Aina. Nous travaillons avec Isabelle Ferrané, notre cliente référente.

L'objectif de cette année est de concevoir et de développer une plateforme mobile équipée de capteurs, capable d'effectuer des tâches manuelles, automatiques et commandées par écrit et à l'oral via une interface utilisateur. Le robot évolue dans un environnement évolutif avec des obstacles de couleurs et de formes différents. Tout le déroulement du projet doit respecter les contraintes définies par le client.

Le dossier de spécifications a pour but de présenter en détail les contraintes à prendre en compte ainsi que les fonctionnalités attendues du système, tout en respectant les besoins exprimés par le client.

I. Présentation du projet

I.1 – Objectifs du document

Le dossier de spécifications a pour objectif de définir l'ensemble des exigences fonctionnelles, techniques et organisationnelles nécessaires à la réalisation de la première phase du Projet Fil Rouge 2025-2026 (PFR1), menée par notre groupe.

Ce document décrit en détail : les fonctionnalités à implémenter, les méthodes et outils utilisés pour répondre aux besoins du cahier des charges, ainsi que les bases de planification et de répartition des tâches au sein de l'équipe.

Il constitue une référence pour les différentes phases de développement, de test et de validation, en vue de produire des modules logiciels intégrables à la plateforme mobile lors de la deuxième phase du projet (PFR2).

De plus, ce dossier comprend : les spécifications techniques de chaque module (conversion des requêtes, analyse d'images, etc.), les interfaces attendues entre les composants du système et les critères de validation associés à chaque fonctionnalité.

Ces spécifications nous permettent d'avoir un fil conducteur pour le projet en respectant les attendus du client et en ayant une vision globale des contraintes techniques et des délais impartis.

I.2 – Objectifs du projet

Ce projet a pour objectif de créer une plateforme mobile robotisée capable de se repérer dans l'espace grâce à différents capteurs (infrarouge, ultrason, caméra, etc.) et de se déplacer en évitant les obstacles et les murs présents dans une pièce.

Le robot possède deux modes de fonctionnement :

- Mode manuel (PFR1 simulation / PFR2 réel) : pilotage via une interface utilisateur permettant à l'opérateur de donner des directions au robot. L'interaction avec le robot peut se faire par écrit au clavier ou vocalement grâce à un micro.
- Mode automatique (PFR2 réel) : déplacement autonome dans un environnement évolutif.

I. Présentation du projet

Ce projet est divisé en deux parties :

- PFR1 : mise en place du projet et développement logiciel avec simulation.
- PFR2 : assemblage et intégration logicielle sur la plateforme mobile.

Ce document porte uniquement sur les tâches à accomplir dans le cadre du PFR1.

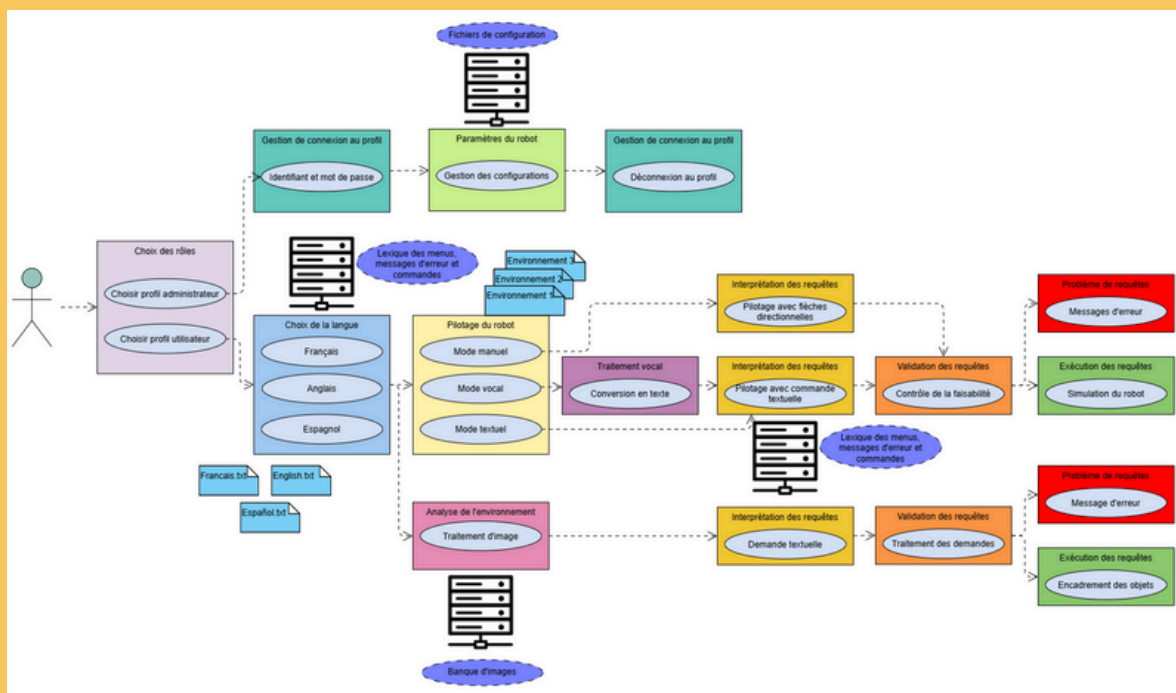


Figure 1 : Diagramme des cas d'utilisations du robot en mode manuel.

Ce diagramme illustre les différentes fonctionnalités du robot. Au démarrage du logiciel, l'utilisateur peut choisir son rôle (utilisateur ou administrateur). Pour accéder au menu de l'administrateur, il faut rentrer le bon identifiant et le mot de passe associé. L'administrateur possède tous les droits sur le robot et peut modifier les fichiers de configurations (plus de détails dans la section II.3 – Interface utilisateur). L'utilisateur peut choisir sa langue, ce qui permet d'adapter le texte des menus dans la langue sélectionnée. Ensuite, l'utilisateur a le choix entre le pilotage du robot et le traitement d'image.

Avant d'effectuer des requêtes de pilotage, l'utilisateur doit choisir dans quel environnement le robot doit évoluer. Trois différentes pièces sont à disposition avec plus ou moins d'obstacles.

I. Présentation du projet

Le pilotage du robot comporte trois modes différents : le mode manuel permet de piloter le robot avec les flèches directionnelles du clavier, le mode vocal permet de donner des instructions vocales au robot et le mode textuel permet de saisir au clavier des instructions dans une zone de texte. Toutes les demandes textuelles et vocales doivent être formulées dans la langue sélectionnée.

Les requêtes demandées au robot sont vérifiées avant qu'elles soient simulées via l'outil turtle. Si les demandes sont jugées impossibles à réaliser par le programme, alors un message d'erreur apparaît à l'écran.

Le traitement d'image est une partie indépendante du projet pour l'instant. L'utilisateur dispose d'une banque d'images représentant des situations de pièces avec plus ou moins d'objets de couleur différente. L'utilisateur peut alors formuler des requêtes permettant de repérer des éléments sur l'image.

II. Fonctionnalités

Le système aura plusieurs modes de pilotage, en premier lieu la requête texte par l'utilisateur, en second lieu, l'interaction vocale puis pour finir le mode traitement d'image pour acquérir des informations sur l'environnement. Nous ajoutons un mode simulation pour prévisualiser les déplacements du système. Parmi les fonctionnalités, nous incluons l'interface utilisateur.

II.1 – Conversions de requêtes (opérateur → système)

II.1.1 – Gestion des requêtes vocales

Dans le cadre du Projet Fil Rouge (PFR1), le développement de la gestion des requêtes vocales vise à assurer la transcription, la conversion et le traitement des commandes vocales afin de les rendre exploitables par le robot.

Les commandes vocales sont d'abord enregistrées et transcrites en texte grâce à un module de reconnaissance vocale fourni par le client.

Cette partie, dédiée à la conversion de la voix en texte, est déjà prise en charge par le module externe et ne fait donc pas partie des tâches à accomplir durant cette phase.

II.1.2 – Conversion requêtes texte en requêtes-commandes

Une fois la commande transcrite sous forme de texte dans l'interface homme-machine (IHM), la fonction de conversion doit transformer cette phrase en une instruction structurée compréhensible par le robot.

Cette étape comprend plusieurs sous-processus :

Analyse linguistique et syntaxique : le programme doit analyser la phrase pour en extraire les éléments significatifs, identifier les groupes de mots utiles (définis dans un fichier de configuration syntaxique) et supprimer les mots superflus. Le résultat est un texte simplifié et structuré.

Génération des commandes : à partir du texte analysé, le système produit des commandes formatées selon le langage du robot, en associant chaque mot-clé à l'action correspondante.

II. Fonctionnalités

Exemples de conversion :

- "avance" → `forward(dist = default_distance)`
- "avance de 2 mètres" → `forward(dist = 2)`
- "trouve la balle rouge" → `where(object = ball, color = red)`
- Gestion des erreurs :
- Si une requête n'est pas comprise, le programme doit afficher un message d'erreur explicite sur le terminal (ex. mot inconnu, faute d'orthographe, mot hors langue choisie, syntaxe incohérente, etc.)

II.1.3 – Gestion des commandes

Afin d'exécuter les instructions successivement, le programme doit analyser la phrase pour identifier les différentes actions par exemple : "avance de 2 mètres et tourne à gauche" → deux commandes distinctes. Il doit pouvoir reconnaître les actions et leurs paramètres associés, par exemple : "avance de 2 mètres" → action = avancer, paramètre = 2 mètres. Et enfin, le programme doit exécuter les actions dans l'ordre logique, en vérifiant la validité de chaque commande avant d'enchaîner sur la suivante

Le système doit aussi être capable de détecter les erreurs de syntaxe, les incohérences de commande et les paramètres manquants.

- Bibliothèque de mots-clés :

Les mots-clés essentiels incluent :

- "avance", "recule", "tourne", "droite", "gauche", "mètre", "mètres", "avant", "arrière", "trouve", "localise", "balle", "cube", "rouge", "vert", etc.

Une bibliothèque de synonymes sera ajoutée afin d'améliorer la flexibilité du système.

Les chiffres seront traités uniquement sous forme numérique (1, 2, 3, ...), et les unités seront abrégées ("m" pour mètre).

Exemples :

- "Avance de 1 m, puis tourne à gauche et avance de 3 m"
- Commandes conditionnelles telles que :
 - "s'il y a une balle bleue, alors fais demi tours" seront considérées comme avancées et intégrées ultérieurement

II. Fonctionnalités

Commandes prises en charge par défaut :

- Commandes de base :
 - Avance → se déplacer vers l'avant.
 - Recule → se déplacer vers l'arrière.
 - Tourne à droite / à gauche → changer de direction.
 - Fais demi-tour → rotation complète.
 - Arrête → immobilisation totale.
- Commandes de déplacement précis :
 - "avance de deux mètres"
 - "avance jusqu'à la balle rouge"
 - "contourne l'obstacle par la droite", etc.
- Commandes avancées :
 - Accélère / Ralentis → ajustement de la vitesse.
 - Change de direction → nouvelle orientation.
 - Retourne au point de départ / Pause / Reprends.
- Commandes complexes (optionnelles) :
 - "contourne l'obstacle sans passer près de la balle rouge"
 - "passe à côté de la balle rouge puis de la balle bleue"
 - "cherche une balle rouge ou bleue"
 - "ne te positionne pas près de la balle verte"
 - "combien vois-tu de balles ?"
 - "fais un zig-zag"

II.1.4 – Scénario de test / erreur

Pour valider la fiabilité du système, une simulation basée sur des commandes Unix est mise en place.

Elle permet de tester le comportement attendu des commandes avant leur intégration finale.

Les tests couvrent différents scénarios :

- Commandes simples
 - Exemple : avance, recule, tourne
 - Problèmes possibles : fautes d'orthographe, synonymes non reconnus, absence de feedback clair.

II. Fonctionnalités

- Commandes conditionnelles
 - Exemple : avance jusqu'à balle_rouge, contourne_obstacle droite
 - Problèmes possibles : objet non détecté, gestion incomplète des conditions, ambiguïté directionnelle
- Enchaînements de commandes
 - Exemple : avance 2 ; tourne_droite 90 ; recule
 - Problèmes possibles : ordre d'exécution incorrect, arrêt inattendu, perte de l'état du robot
- Gestion des erreurs et feedback utilisateur
 - Tout échec doit générer un message clair (ex. : Erreur : paramètre distance manquant)
 - Les erreurs doivent être journalisées (logs) afin de faciliter le débogage et l'amélioration continue

Pour avoir une prévisualisation des déplacements du système, nous intégrons une simulation du déplacement de système faisant suite aux requêtes texte.

II.2 – Simulation des déplacements

Ce module présente en détail toutes les fonctionnalités prévues pour la simulation des déplacements, c'est-à-dire la gestion de l'activation des déplacements du système par l'utilisateur ainsi que la gestion des cas d'erreurs prévues par le système.

La simulation s'effectuera à l'aide de la bibliothèque turtle de python.

II.2.1 – Déplacement par commande utilisateur

L'utilisateur doit pouvoir gérer le déplacement du robot de trois façons différentes : commande manuelle, commande textuelle et commande vocale.

Chaque commande donnée par l'utilisateur devra être analysée et interprétée par le système pour effectuer l'action adéquate. Les requêtes manuelles et textuelles seront directement traitées par le système après que l'utilisateur ait appuyé sur la commande ou validé sa saisie. La requête vocale sera dans un premier temps retransmise en texte pour ensuite être traitée par le système.

II. Fonctionnalités

II.2.2 Gestion de l'environnement

Le système évoluant dans un environnement fermé, trois environnements sont à prendre en considération :

- La pièce est vide : Un environnement vide est reconnu, seules les requêtes de déplacement sans traitement d'image seront possibles.
- La pièce possède des obstacles de même type (couleur et forme) : Le module de traitement d'image s'ajoute aux déplacements pour pouvoir traiter des requêtes plus complexes sans différencier les obstacles.
- La pièce possède des obstacles de types différents : La prise en compte de plusieurs types d'obstacles s'ajoute pour pouvoir traiter l'entièreté des requêtes possibles.

II.2.3 Gestion des erreurs

Avant tout déplacement, le système devra analyser si celui-ci est possible et envoyer un retour à l'utilisateur en cas d'impossibilité.

- Un fichier log devra enregistrer et répertorier toutes les erreurs survenues.

Les erreurs de déplacement seront segmentées en deux parties distinctes :

1. Requête utilisateur impossible à interpréter :

- La commande entrée par l'utilisateur n'est pas reconnue
- La commande ne correspond pas à l'utilisation prévue du système ou n'a pas de sens

2. Objectif de la requête impossible à satisfaire :

- La requête a bien été interprétée, mais le système ne peut pas l'exécuter (objet non trouvable ou obstacle sur le chemin)

- Un retour du type d'erreur devra être fourni à l'utilisateur, qui pourra relancer sa requête sans relancer le système.

Afin de fluidifier le déplacement de l'utilisateur entre les différentes fonctionnalités, nous introduisons dès maintenant l'interface utilisateur.

II. Fonctionnalités

II.3 – Interface utilisateur

L'interface homme machine (IHM) du robot permet à l'utilisateur de naviguer à travers les différentes fonctionnalités proposées. Deux types d'interface sont mis à disposition : une interface pour l'utilisateur et une interface réservée à l'administrateur. Dès la mise en fonctionnement du système, l'utilisateur doit choisir son rôle.

II.3.1 – Utilisateur : gestion des langues

L'IHM du robot est disponible en plusieurs langues. À l'allumage, l'utilisateur doit sélectionner sa langue de préférence.

Langues	Niveaux de développement
Français, Anglais	1
Espagnol	2

Une fois la langue sélectionnée, l'ensemble du système adapte son interface, et toutes les requêtes textuelles et vocales doivent être formulées dans la langue choisie.

II.3.2 – Utilisateur : modes de fonctionnement

Le système dispose de deux utilisations : le pilotage du robot et le traitement d'image.

Pour le pilotage du robot, trois modes d'instructions sont à disposition : le mode manuel, le mode vocal et le mode textuel. L'utilisateur doit sélectionner un des trois modes pour pouvoir déplacer le robot dans sa simulation.

L'interface du mode vocal dispose d'une zone de texte qui permet de visualiser les instructions données par l'utilisateur. Celui-ci peut ainsi voir directement si le système a bien reçu ses demandes.

L'interface du mode textuel est une zone de texte où l'utilisateur peut saisir au clavier ses instructions.

II. Fonctionnalités

Pour le traitement d'image, l'utilisateur doit sélectionner l'image souhaitée mise à disposition via un champ de texte. Une fois l'image sélectionnée, celle-ci apparaît et l'utilisateur peut donner des requêtes permettant au système de repérer des objets associés à leurs couleurs.

II.3.3 – Administrateur : connexion

Pour accéder à l'interface de l'administrateur, un identifiant et un mot de passe sont demandés. L'utilisateur dispose de trois tentatives pour se connecter. Si celui-ci échoue, alors un temps d'attente de 1 minute survient avant de recommencer.

II.3.4 – Administrateur : configuration

Le système offre des options de paramétrage permettant à l'utilisateur de configurer le traitement d'images (bits de quantification, plages de couleurs, détection des objets), la langue et le lexique pour la conversion des requêtes textuelles en commandes, ainsi que la génération d'un fichier de log contenant l'historique des commandes exécutées, succès et échecs inclus.

Toutes ces configurations sont sous des fichiers visualisables par l'administrateur. La navigation se fait par la sélection de catégories de paramètres (citées au-dessus) facilitant ainsi la lisibilité.

Les paramètres de traitement d'image sont les suivants : modification du bit de quantification, modification des formes et modification des couleurs reconnues par le système.

Les paramètres des langues et du lexique sont les suivants : modification des mots traduits, ajout de nouveaux mots et ajout de nouveaux termes interprétables.

Tous les fichiers sont de type CSV, facilitant ainsi le développement du code.

Ces configurations permettent aux fonctionnalités développées dans le PFR1 de rester flexibles pour d'autres scénarios ou situations futures.

II. Fonctionnalités

II.3.5 – Scénarios de tests et gestion d’erreurs

Chaque requête et action utilisateur est validée avant exécution. Lorsqu’une situation est impossible à réaliser ou qu’une erreur survient lors de l’utilisation du robot, des messages d’erreurs (textuels) sont transmis à l’opérateurs via l’interface.

Voici une liste de la gestion d’erreurs du robot et des scénarios possibles :

- Erreurs lors de l’authentification :
 - Si le nom d’utilisateur est incorrect : “Nom d’utilisateur inconnu. Veuillez réessayer.”
 - Si le mot de passe est incorrect : message d’erreur et nombre limité de tentatives (3 essais). Après échec répété, l’accès est temporairement bloqué pour des raisons de sécurité. Une clé de sécurité doit être saisie pour débloquer le système.
- Erreurs de saisie des requêtes :

Le système vérifie que chaque commande textuelle ou vocale est :

 - Syntaxiquement correcte : correspond aux commandes prévues.
 - Exécutable : réalisation des requêtes dans la configuration actuelle (absence d’obstacles, objets détectés, etc.)
- En cas d’erreur :
 - Le système informe l’utilisateur via synthèse vocale ou message texte :
 - “Commande inconnue”, “Déplacement impossible”, “Objet introuvable”
 - Si possible, le système propose des alternatives ou demande une clarification :
 - “Commande non reconnue. Veuillez réessayer.”

Chaque erreur est enregistrée dans le fichier de log pour la traçabilité et les tests futurs.

Cette liste garantit que le robot réagit de manière prévisible aux erreurs, que l’utilisateur est informé et guidé, et que toutes les anomalies sont renseignées dans un fichier pour un suivi et débogage futur.

Lors des essais du robot, cette liste nous permettra d’établir un suivi des tests à réaliser pour permettre une utilisation fiable par la suite du projet.

II. Fonctionnalités

Cette dernière fonctionnalité , le traitement d'image, permet de repérer des formes et des couleurs sur des images représentant un environnement pour le robot.

II.4 – Traitement d'images

II.4.1 – Objectif global

Créer un module C (ou C + Python) capable de charger une image couleur (format simple : PPM, BMP ou PNG converti). Il devra extraire les informations de couleur et détecter les zones homogènes (objets). Caractériser chaque objet (forme, couleur dominante, position, taille) et retourner les coordonnées du centre et/ou de la boîte englobante.

II.4.2 – Architecture modulaire proposée

Proposition d'organisation générale du code en modules C :

- "image" pour la lecture et structure interne
 - "détection" pour l'algorithme de détection des zones homogènes
 - "analyse" pour la caractérisation des couleurs, formes, tailles
-

II.4.3 – Étapes de traitement d'image

Étape 1 – Chargement et représentation interne

Le module doit lire une image simple. (PPM recommandé car facile à parser sans bibliothèque externe). En effet, le format PPM est un format texte très basique contenant uniquement les valeurs des pixels (rouge, vert, bleu). On peut donc le lire avec `fscanf`, sans avoir besoin d'une bibliothèque comme OpenCV.

Étape 2 – Prétraitement

Proposition : conversion RGB → HSV (plus stable pour détecter les couleurs indépendamment de la luminosité).

II. Fonctionnalités

En traitement d'image, chaque pixel est représenté par une couleur codée selon un modèle :

- RGB (*Rouge, Vert, Bleu*) → modèle "écran", brut, lié à l'intensité lumineuse des couleurs primaires.
 - En RGB, une même couleur vue sous des conditions de lumière différentes aura des valeurs très variables.
 - Exemple : un rouge éclairé et un rouge dans l'ombre auront des triplets (R,G,B) très différents.
- HSV (*Teinte, Saturation, Valeur*) → modèle "perceptif", proche de la perception humaine.
 - Hue (H) : la teinte (rouge, vert, bleu, jaune...), exprimée en degrés (0° à 360°).
 - Saturation (S) : la "pureté" de la couleur (de 0 à 1).
 - Value (V) : la luminosité (de 0 à 1).

En HSV, la teinte (H) reste constante même si la luminosité change. Cela permet une détection de couleur plus robuste avec moins de possibilité d'erreur dû aux variations d'éclairage.

Étape 3 — Solution de filtrage

L'objectif ici est de réduire le bruit tout en préservant les vraies structures.

Proposition 1 : Filtrage par moyenne (moyennage)

On remplace chaque pixel par la moyenne de ses 9 voisins (fenêtre 3×3). Cela permet de lisser les variations rapides et ainsi de réduire le bruit sans perte de détails.

Exemple :

```
120 122 118
123 255 119
121 120 122
```

Moyenne ≈ 135 → le pixel central bruité (255) devient 135.

Proposition 2 : Filtrage par médiane

On prend la valeur médiane parmi les 9 pixels. Ce filtrage est plus robuste contre le bruit impulsif (pixels isolés très clairs ou sombres).

II. Fonctionnalités

Exemple :

Valeurs triées : 118, 119, 120, 120, 121, 122, 122, 123, 255 → médiane = 121

→ Le pixel bruité 255 devient 121 : nettoyage efficace sans flou excessif.

Étape 4 — Détection d'objets

Dans cette étape, le programme commence par créer, pour chaque couleur recherchée, une image simplifiée appelée *masque binaire*. Ce masque agit comme un filtre qui met en évidence uniquement les zones contenant la couleur visée et ignore le reste. Une fois ce repérage effectué, le logiciel utilise un principe de remplissage — semblable à un pot de peinture numérique — pour regrouper les pixels voisins d'une même couleur. Ce procédé, appelé *flood fill*, permet de former des zones continues correspondant à des objets ou éléments distincts de l'image.

Pour chaque région détectée on sera alors capable d'avoir :

- Le nombre de pixels
- Les coordonnées min/max → boîte englobante
- Les coordonnées moyennes → centre

II.4.4 – Modules / fonctions à écrire soi-même

Module	Contenu	Pourquoi
image	Lecture / écriture d'images PPM	Manipulation bas niveau
couleur	Conversion RGB ↔ HSV, comparaison de teintes	Comprendre la couleur
segmentation	Flood fill, étiquetage de composantes	Algorithmique impérative
analyse	Calcul centre, forme, taille	Traitement numérique
résultats	Écriture des résultats	Intégration future (PFR2)

III. Organisation

III.1 – Outils utilisés

Afin d'assurer le bon déroulement du Projet Fil Rouge, différents outils logiciels ont été sélectionnés pour couvrir l'ensemble des besoins en développement, gestion de version et planification.

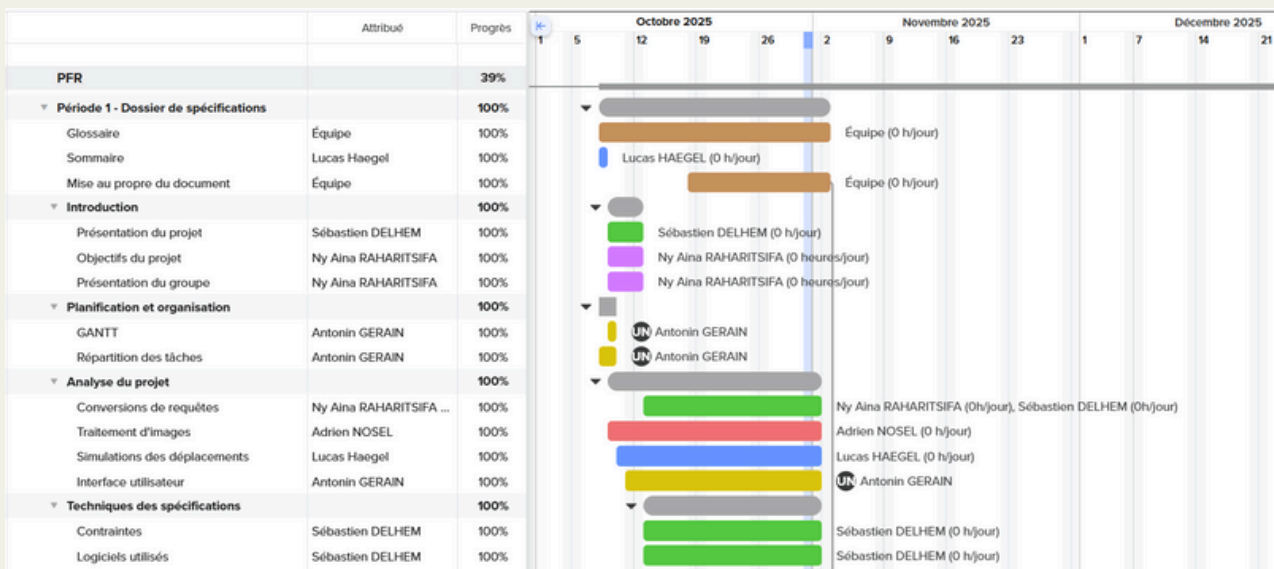
Catégorie d'outil	Nom	Détails / Version
Systèmes d'exploitation	Windows 11	Utilisé pour le développement et les tests sur environnement graphique.
	Debian (UNIX)	Utilisé pour les tests en environnement Linux et la compatibilité multiplateforme.
	macOS Sequoia	Utilisé pour la vérification de la portabilité du projet.
Environnements de développement	Visual Studio Code	Version 1.105.0 – Utilisé pour le développement principal. • Python 3.13.6 • C C23 • Jupyter 7.4.7
Gestionnaires de configuration Web	GitHub	Version 3.6 – Plateforme d'hébergement du code et suivi collaboratif.
	Git	Version 2.51.0 – Gestion locale des versions et synchronisation avec GitHub.

III. Organisation

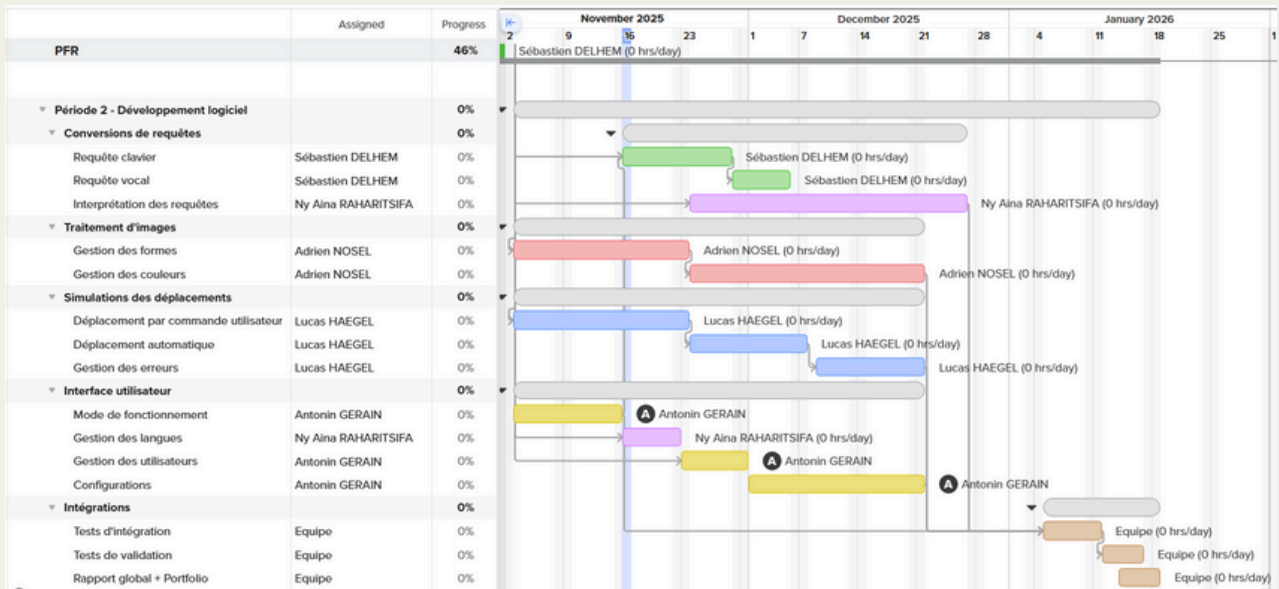
Outils de gestion de projet	Trello	Utilisé pour la planification des tâches, le suivi d'avancement et la répartition du travail au sein de l'équipe.
Virtualisation Simulation	VirtualBox	Version 7.1.12 – Environnement de travail utilisant Debian.

III.2 – Planning prévisionnel

Pour planifier les tâches du projet, nous avons utilisé le site internet TeamGantt. Ce site permet de réaliser des diagrammes de GANTT personnalisables sans utilisation de l'intelligence artificielle. Le site permet d'affecter des tâches aux différents membres du groupe et de compléter le pourcentage de réalisation des ces dernières. Le logiciel a une fonctionnalité de calcul de temps de travail, il ne faut pas le prendre en compte ici.



III. Organisation



III.3 – Répartition des rôles et des responsabilités

Nom et prénom	Rôle dans le projet	Responsabilité des tâches
DELHEM Sébastien	Chargé de communication, développeur	Conversion de requête
GERAIN Antonin	Chef de projet, développeur	Interface utilisateur
HAEGEL Lucas	Développeur	Simulation des déplacements
NOSEL Adrien	2eme chargé de communication (remplacement), développeur	Traitement d'image
RAHARITSIFA Ny Aina	Développeur	Conversion de requête

